Formula 1 World Championship Database: A PostgreSQL System for Motorsport Analytics (1950–2024)

Sourabh Raja
University at Buffalo
sourabhr

Sphoorthy Selvaraj
University at Buffalo
selvara3

Sindhu Shrishailappa Jumanal
University at Buffalo
sjumanal

TABLE I
PROJECT INFORMATION

Field	Value
Title	F1 Race Analytics Database
Course	CSE 560 — Data Models and Query Languages
Term	Spring 2026, University at Buffalo
Phase	Phase 1 — Project Proposal
Due Date	March 15, 2026
Dataset	Formula 1 World Championship (Kaggle)
DBMS	PostgreSQL
Records	700,000+ across 14 tables
Coverage	1950–2024 (75 seasons)

TABLE II
TEAM CONTRIBUTION SUMMARY

Name	Contribution (Phase 1 & Phase 2)	%
Sphoorthy Selvaraj	Ph.1: Schema Design, E/R; Ph.2: SQL Queries (T5)	33%
Sourabh Raja	Ph.1: Data Acquisition, BCNF; Ph.2: Stored Procedures, Triggers (T6)	33%
Sindhu S Jumanal	Ph.1: Normalization Analysis; Ph.2: Transactions, Indexing (T7)	34%

***Abstract*—This report presents the Phase 1 proposal for the F1 Race Analytics Database, a PostgreSQL-based relational database built on the Formula 1 World Championship dataset spanning 75 years of racing history (1950–2024). The system integrates 14 interrelated tables covering circuits, drivers, constructors, race results, lap times, pit stops, qualifying sessions, and championship standings, totalling over 700,000 records. This proposal covers the problem domain, target users, entity-relationship design, relational schema, and a complete BCNF normalization analysis verified directly from the source CSV data.**

I. PROJECT DETAILS

Project: Formula 1 World Championship Database
Course: CSE 560 – Data Models and Query Language, Spring 2026
Institution: University at Buffalo, CSE Department
Instructor: Prof. Shamsad Parvin, PhD

A. Project Information

Table I summarizes the key project metadata.

B. Team Members and Contribution

Table II lists all team members, their UB IDs, and contribution. Each member contributed equally to all parts of the project.

at a genuinely interesting intersection of engineering, statistics, and sports history. The data is highly relational by nature, races connect to circuits, drivers connect to constructors, lap times connect to both races and drivers and it spans 74 years, which makes trend analysis meaningful rather than just a point-in-time snapshot.

A. Title and Core Problem

Title: *F1 Race Analytics Database: A PostgreSQL System for Historical Formula 1 Performance Analysis*

The problem lies in the absence of a unified, queryable relational system over the complete F1 historical record. Existing approaches rely on spreadsheets that cannot support multi-table joins, enforce referential integrity, or handle the data volume involved. This database which we propose is designed to answer the following questions:

- Which driver has the highest average points-per-race over a full career?
- Which constructor showed the greatest improvement in standings between any two consecutive seasons?
- How does pit stop duration correlate with final finishing position?
- How have fastest lap times at circuits ?
- What is the statistical relationship between qualifying position and race result?

B. Background

The F1 dataset used in this project contains 14 interrelated CSV files covering every race from 1950 to 2024. The data spans over multiple entity types such as drivers, constructors, circuits, race events, lap times, pit stops, and standings with

TABLE III
TARGET USER GROUPS

User	Access	Primary Use
F1 Analyst	Read-only	Career stats, lap time trends
Journalist	Read-only	Historical comparisons, race reports
Race Strategist	Read/Write	Pit stop and qualifying analysis
DBA	Full	Data loading, schema, maintenance

complex many-to-many relationships between them. A spreadsheet cannot represent these relationships without redundancy, update anomalies and it is is cumbersome to fetch results from it.

There is a significant problem without the database. First, F1 analytics is an active domain used by teams, journalists, and broadcasters to derive competitive insights. Second, the dataset scale (700,000+ records) exceeds what any spreadsheet tool can manage efficiently. Third, the multi-entity nature of the data, a single race involves drivers, constructors, a circuit, lap times per driver per lap, pit stops, and a result record , demands relational modelling with enforced foreign key constraints and normalization.

C. Why a Database Instead of Excel

A spreadsheet tool such as Microsoft Excel is fundamentally unsuitable for this use case for the following reasons:

The dataset has 589,081 lap time records alone. A spreadsheet cannot handle that scale, cannot enforce referential integrity across 14 linked tables, and cannot execute the kind of multi-table transaction that inserting a complete race weekend requires. PostgreSQL gives us foreign-key constraints, ACID transactions, stored procedures, triggers, and indexed lookups

D. Contribution and Potential

This project delivers a fully normalized, query-ready PostgreSQL database over the complete F1 historical record. Its contributions to the domain are:

- A verified BCNF-compliant schema for all 14 tables, with functional dependencies derived directly from the source data.
- A reusable SQL foundation for F1 analytics that supports both historical queries and post-race updates.
- Demonstration of advanced SQL features including window functions, correlated subqueries, stored procedures and triggers.

III. TARGET USERS

The database serves four user groups with different access needs, summarized in Table III.

A. Primary Users

F1 Analysts query multi-table joins across `lap_times`, `results`, and `circuits` to compare driver performance across track types and seasons.

Sports Journalists retrieve head-to-head statistics and historical records during race weekends using aggregation queries on `results` and `driver_standings`.

B. Secondary Users

Race Strategists query `pit_stops` and `lap_times` during live races to evaluate undercut opportunities, with limited write access for session notes.

IV. TASK 2: SCHEMA DESIGN

A. E/R Diagram

Figure 1 shows the entity-relationship diagram. Tables with orange-red title bars are the two that contain BCNF violations in the initial schema (discussed in Task 4). Double-bordered tables are weak entities with composite primary keys.

The schema has four layers. DRIVER, CONSTRUCTOR, CIRCUIT, and SEASON are master entities that change rarely and are referenced by everything else. RACE and RESULT form the central table where most queries land. LAP_TIME, PIT_STOP, QUALIFYING, and SPRINT_RESULT provide granular per-event detail. The standings tables track championship progress race by race.

B. Relational Schema

All 14 tables are described below. Column widths in the attribute tables follow the convention: **PK** = primary key in red, FK = foreign key in blue.

1) SEASONS — 75 rows*:*

Attribute	Type	Null	Notes
year	INT	No	**PK**; 1950–2024
url	TEXT	Yes	Wikipedia season link

Year is the primary key.

2) CIRCUITS — 77 rows*:*

Attribute	Type	Null	Notes
circuitId	SERIAL	No	**PK**
circuitRef	VARCHAR(50)	No	Candidate key; URL slug
name	VARCHAR(100)	No	Candidate key; circuit name
location	VARCHAR(100)	Yes	City or region
country	VARCHAR(60)	Yes	Country name
lat	NUMERIC(8,5)	Yes	Latitude coordinate
lng	NUMERIC(8,5)	Yes	Longitude coordinate
alt	INT	Yes	Altitude in metres
url	TEXT	Yes	Wikipedia circuit link

Three candidate keys are verified: circuitId, circuitRef, and name are each unique across all 77 rows. FK action: circuitId is referenced by RACE with RESTRICT on delete.

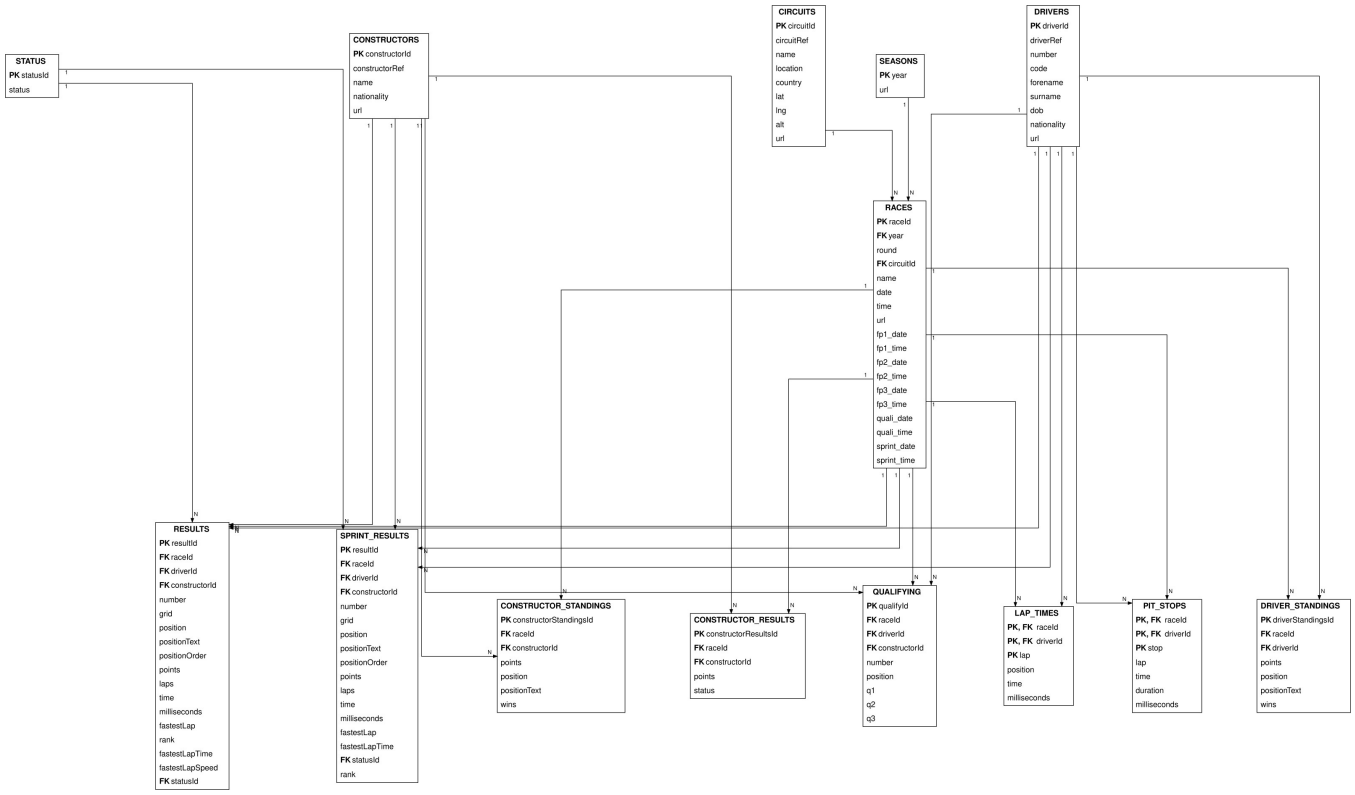


Fig. 1. F1 Database E/R Diagram.

3) CONSTRUCTORS — 212 rows*:*

Attribute	Type	Null	Notes
constructorId	SERIAL	No	**PK**
constructorRef	VARCHAR(50)	No	Candidate key; URL slug
name	VARCHAR(100)	No	Candidate key; team name
nationality	VARCHAR(50)	Yes	e.g., British (86 teams)
url	TEXT	Yes	Wikipedia team link

Three candidate keys: constructorId, constructorRef, name. FK action: constructorId is referenced by RESULT, QUALIFYING, and the standings tables with CASCADE on delete.

4) DRIVERS — 861 rows*:*

Attribute	Type	Null	Notes
driverId	SERIAL	No	**PK**
driverRef	VARCHAR(50)	No	Candidate key; URL slug
number	INT	Yes	Perm. number; 802/861 null
code	CHAR(3)	Yes	NOT a CK; 7 codes reused
forename	VARCHAR(50)	No	First name
surname	VARCHAR(50)	No	Surname
dob	DATE	Yes	Date of birth
nationality	VARCHAR(50)	Yes	e.g., British (166 drivers)
url	TEXT	Yes	Wikipedia driver link

Only driverId and driverRef are candidate keys. The code column looks like a unique identifier but seven codes (ALB, MSC, MAG, DOO, VER, BIA, HAR) were reused by different drivers across different seasons. FK action: driverId is referenced by RESULT, LAP_TIME, PIT_STOP, QUALIFYING.

5) STATUS — 139 rows*:*

Attribute	Type	Null	Notes
statusId	SERIAL	No	**PK**
status	VARCHAR(50)	No	Candidate key; e.g., Finished

FK : statusId referenced by RESULT and SPRINT_RESULT with RESTRICT on delete.

6) RACES — 1,125 rows*:*

Attribute	Type	Null	Notes
raceId	SERIAL	No	**PK**
year	INT	No	FK → SEASONS; RESTRICT
round	INT	No	Round number in season
circuitId	INT	No	FK → CIRCUITS; RESTRICT
name	VARCHAR(100)	No	Grand Prix name
date	DATE	No	Race date
time	TIME	Yes	Race start time
url	TEXT	Yes	Wikipedia race link
fp1_date	DATE	Yes	Practice 1 date
fp2_date	DATE	Yes	Practice 2 date
fp3_date	DATE	Yes	Practice 3 date
quali_date	DATE	Yes	Qualifying date
sprint_date	DATE	Yes	Sprint race date

The composite *{*year, round*}* is a candidate key.

7) RESULTS — 26,759 rows*:*

Attribute	Type	Null	Notes
resultId	SERIAL	No	**PK** (only true key)
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
number	INT	Yes	Car number used this race
grid	INT	Yes	Starting grid position
position	INT	Yes	Finish position; 41% null
positionText	VARCHAR(5)	Yes	Text version of position
positionOrder	INT	No	Always-populated sort field
points	NUMERIC(5,2)	No	Championship points scored
laps	INT	Yes	Laps completed
time	VARCHAR(20)	Yes	Finish time string
milliseconds	INT	Yes	Finish time in ms
fastestLap	INT	Yes	Lap number of fastest lap
rank	INT	Yes	Fastest lap rank in race
fastestLapTime	VARCHAR(10)	Yes	e.g., 1:27.452
fastestLapSpeed	NUMERIC(7,3)	Yes	Speed in km/h
statusId	INT	No	FK → STATUS; RESTRICT

resultId is the only valid primary key.

8) QUALIFYING — 10,494 rows*:*

Attribute	Type	Null	Notes
qualifyId	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
number	INT	Yes	Car number used
position	INT	Yes	Qualifying result
q1	VARCHAR(10)	Yes	Q1 lap time
q2	VARCHAR(10)	Yes	Q2 time
q3	VARCHAR(10)	Yes	Q3 time

*{*raceId, driverId*}* IS a candidate key here

9) LAP_TIMES — 589,081 rows (weak entity)*:*

Attribute	Type	Null	Notes
raceId	INT	No	**PK** + FK → RACES; CASCADE
driverId	INT	No	**PK** + FK → DRIVERS; CASCADE
lap	INT	No	**PK**; lap number
position	INT	Yes	Track position end of lap
time	VARCHAR(10)	Yes	Lap time, e.g., 1:27.452
milliseconds	INT	Yes	Same time in ms

Composite PK *{*raceId, driverId, lap*}* is unique across all rows

10) PIT_STOPS — 11,371 rows*:*

Attribute	Type	Null	Notes
raceId	INT	No	**PK** + FK → RACES; CASCADE
driverId	INT	No	**PK** + FK → DRIVERS; CASCADE
stop	INT	No	**PK**; pit stop number
lap	INT	No	Lap on which stop occurred
time	TIME	Yes	Time of day of stop
duration	VARCHAR(10)	Yes	Duration string
milliseconds	INT	Yes	Duration in ms

Composite PK *{*raceId, driverId, stop*}* verified across all rows

11) DRIVER_STANDINGS — 34,863 rows*:*

Attribute	Type	Null	Notes
driverStandId	SERIAL	No	**PK**
driverStandingsId	SERIAL		
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
points	NUMERIC(6,2)	No	Cumulative points
position	INT	Yes	Championship position
wins	INT	No	Cumulative wins
is_disqualified	BOOLEAN	No	Default FALSE; see Task 4

Note: positionText was removed during BCNF decomposition and replaced with `is_disqualified`*.*

12) CONSTRUCTOR_STANDINGS — 13,391 rows*:*

Attribute	Type	Null	Notes
constrStandId*	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
points	NUMERIC(6,2)	No	Cumulative points
position	INT	Yes	Championship position
wins	INT	No	Cumulative wins
is_excluded	BOOLEAN	No	Default FALSE; see Task 4

Note: positionText removed during BCNF decomposition. Replaced with `is_excluded`*.*

13) SPRINT_RESULTS — 360 rows*:*

Attribute	Type	Null	Notes
resultId	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
driverId	INT	No	FK → DRIVERS; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
position	INT	Yes	Sprint finish position
positionText	VARCHAR(5)	Yes	Text version
points	NUMERIC(4,2)	No	Max 8 points for sprint win
laps	INT	Yes	Laps completed
statusId	INT	No	FK → STATUS; RESTRICT

*{*raceId, driverId*}* IS a candidate key here (360 unique pairs verified). Unlike the main results table, no constructor swaps occur in sprint races. 18 sprint events from 2021–2024.

14) CONSTRUCTOR_RESULTS — 12,625 rows*:*

Attribute	Type	Null	Notes
constrResultsId	SERIAL	No	**PK**
raceId	INT	No	FK → RACES; CASCADE
constructorId	INT	No	FK → CONSTRUCTORS; CASCADE
points	NUMERIC(5,2)	Yes	Points this race
status	CHAR(1)	Yes	'D' = disqualified; else null

V. Task 3: Dataset Acquisition

A. Source and Access

The data comes from the Ergast Motor Racing Developer API [1], which has maintained Formula 1 records since 2005. Rohan Rao packaged the full Ergast database as 14 CSV files on Kaggle, updated through January 2025 [2].

Download link: https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020

No account or special access is required. The dataset is published under an open licence for non-commercial use.

B. BCNF Analysis

For each relation, we verify that every non-trivial functional dependency $X \rightarrow Y$ has X as a superkey. This is confirmed by computing X^+ from the source data.

1) seasons:

- **Attributes:** {year, url}
- **FDs:** $year \rightarrow url$, $url \rightarrow year$
- **Closures:** $\{year\}^+ = R$, $\{url\}^+ = R$
- **Candidate Keys:** {year}, {url}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

2) circuits:

- **Attributes:** {circuitId, circuitRef, name, location, country, lat, lng, alt, url}
- **FDs:** $circuitId \rightarrow all$, $circuitRef \rightarrow all$, $name \rightarrow all$
- **Closures:** $\{circuitId\}^+ = R$, $\{circuitRef\}^+ = R$, $\{name\}^+ = R$
- **Candidate Keys:** {circuitId}, {circuitRef}, {name}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

3) drivers:

- **Attributes:** {driverId, driverRef, number, code, forename, surname, dob, nationality, url}
- **FDs:** $driverId \rightarrow all$, $driverRef \rightarrow all$
- **Closures:** $\{driverId\}^+ = R$, $\{driverRef\}^+ = R$
- **Candidate Keys:** {driverId}, {driverRef}
- **Note:** code and number are not candidate keys — 7 codes are reused across eras, and 802/861 numbers are NULL.
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

4) constructors:

- **Attributes:** {constructorId, constructorRef, name, nationality, url}
- **FDs:** $constructorId \rightarrow all$, $constructorRef \rightarrow all$, $name \rightarrow all$
- **Closures:** $\{constructorId\}^+ = R$, $\{constructorRef\}^+ = R$, $\{name\}^+ = R$
- **Candidate Keys:** {constructorId}, {constructorRef}, {name}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

5) status:

- **Attributes:** {statusId, status}
- **FDs:** $statusId \rightarrow status$, $status \rightarrow statusId$
- **Closures:** $\{statusId\}^+ = R$, $\{status\}^+ = R$
- **Candidate Keys:** {statusId}, {status}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

6) races:

- **Attributes:** {raceId, year, round, circuitId, name, date, time, url, fp1_date, fp2_date, fp3_date, quali_date, sprint_date}
- **FDs:** $raceId \rightarrow all$, $(year, round) \rightarrow all$
- **Closures:** $\{raceId\}^+ = R$, $\{year, round\}^+ = R$
- **Candidate Keys:** {raceId}, {year, round}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

7) results:

- **Attributes:** {resultId, raceId, driverId, constructorId, grid, position, positionText, positionOrder, points, laps, time, milliseconds, fastestLap, rank, fastestLapTime, fastestLapSpeed, statusId}
- **FDs:** $resultId \rightarrow all$
- **Closures:** $\{resultId\}^+ = R$
- **Candidate Keys:** {resultId} only
- **Note:** $(raceId, driverId)$ is not a candidate key , 176 rows exist where a driver raced under two constructors in the same race.
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

8) qualifying:

- **Attributes:** {qualifyId, raceId, driverId, constructorId, number, position, q1, q2, q3}
- **FDs:** $qualifyId \rightarrow all$, $(raceId, driverId) \rightarrow all$
- **Closures:** $\{qualifyId\}^+ = R$, $\{raceId, driverId\}^+ = R$
- **Candidate Keys:** {qualifyId}, {raceId, driverId}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

9) lap_times:

- **Attributes:** {raceId, driverId, lap, position, time, milliseconds}
- **FDs:** $(raceId, driverId, lap) \rightarrow position, time, milliseconds$

- **Closures:** $\{raceId, driverId, lap\}^+ = R$
- **Candidate Keys:** {raceId, driverId, lap}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

10) pit_stops:

- **Attributes:** {raceId, driverId, stop, lap, time, duration, milliseconds}
- **FDs:** $(raceId, driverId, stop) \rightarrow lap, time, duration, milliseconds$
- **Closures:** $\{raceId, driverId, stop\}^+ = R$
- **Candidate Keys:** {raceId, driverId, stop}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

11) constructor_results:

- **Attributes:** {constructorResultsId, raceId, constructorId, points, status}
- **FDs:** $constructorResultsId \rightarrow all$, $(raceId, constructorId) \rightarrow all$
- **Closures:** $\{constructorResultsId\}^+ = R$, $\{raceId, constructorId\}^+ = R$
- **Candidate Keys:** {constructorResultsId}, {raceId, constructorId}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

12) sprint_results:

- **Attributes:** {resultId, raceId, driverId, constructorId, position, positionText, points, laps, statusId}
- **FDs:** $resultId \rightarrow all$, $(raceId, driverId) \rightarrow all$
- **Closures:** $\{resultId\}^+ = R$, $\{raceId, driverId\}^+ = R$
- **Candidate Keys:** {resultId}, {raceId, driverId}
- **Conclusion:** All FDs have a superkey on the LHS. **In BCNF.**

13) driver_standings — BCNF Violation:

- **Attributes:** {driverStandingsId, raceId, driverId, points, position, positionText, wins}
- **FDs:** $driverStandingsId \rightarrow all$, $(raceId, driverId) \rightarrow all$, $position \rightarrow positionText$
- **Closure of violating determinant:** $\{position\}^+ = \{position, positionText\} \neq R$
- **Candidate Keys:** {driverStandingsId}, {raceId, driverId}
- **Violation:** $position \rightarrow positionText$ violates BCNF as position is not a superkey.

Resolution: `positionText` is dropped and replaced with `is_disqualified` BOOLEAN (default FALSE), capturing the only meaningful non-numeric case. The resulting table is in BCNF as all remaining FDs have a superkey on the LHS.

14) constructor_standings — BCNF Violation:

- **Attributes:** {constructorStandingsId, raceId, constructorId, points, position, positionText, wins}
- **FDs:** $constructorStandingsId \rightarrow all$, $(raceId, constructorId) \rightarrow all$, $position \rightarrow positionText$
- **Closure of violating determinant:** $\{position\}^+ = \{position, positionText\} \neq R$
- **Candidate Keys:** {constructorStandingsId}, {raceId, constructorId}
- **Violation:** $position \rightarrow positionText$ violates BCNF as position is not a superkey. Verified: 17 rows have positionText = 'E' (Excluded) while position holds a numeric value.

Resolution: `positionText` is dropped and replaced with `is_excluded` BOOLEAN (default FALSE), capturing the only meaningful non-numeric case. The resulting table is in BCNF as all remaining FDs have a superkey on the LHS.

Final schema:

```
DRIVER_STANDINGS(driverStandingsId, raceId,
    driverId, points, position, wins,
        blueis_disqualified)
```

FDs after decomposition: $\{driverStandingsId\} \rightarrow \{all\}$; $\{raceId, driverId\} \rightarrow \{all\}$. Both are superkeys. **BCNF confirmed.**

Lossless: positionText is exactly reconstructable from `position` and `is_disqualified`. No information is lost.

15) CONSTRUCTOR_STANDINGS:

Violation: $\{position\} \rightarrow \{positionText\}$ fails because position = 11 maps to both "11" (normal) and "E" (Excluded) in 17 rows for constructorId = 1. Same structural problem as driver standings.

Initial schema:

```
CONSTRUCTOR_STANDINGS(constructorStandingsId,
  raceId, constructorId, points, position,
          redpositionText, wins)
```

Decomposition: Remove `positionText`. Add `is_excluded BOOLEAN DEFAULT FALSE`. Reconstructable as:

```
CASE WHEN is_excluded THEN 'E' ELSE CAST(position AS
                VARCHAR) END
```

Final schema:

```
CONSTRUCTOR_STANDINGS(constructorStandingsId,
raceId, constructorId, points, position, wins,
              blueis_excluded)
```

FDs after decomposition: $\{constructorStandingsId\} \rightarrow \{all\}$; $\{raceId, constructorId\} \rightarrow \{all\}$. **BCNF confirmed.**
Lossless: positionText reconstructable from `position` and `is_excluded`. No information lost.

C. Final Schema BCNF Summary

After decomposition, all 14 relations satisfy BCNF. Twelve of fourteen tables were already in BCNF. The two violations shared the same root cause and were both resolved through lossless decomposition that preserves all original information.

V. TASK 5: SQL QUERIES

With the schema fully built and loaded, the database was tested using twelve SQL queries from queries.sql. The queries cover the required INSERT, DELETE, UPDATE, and SELECT categories, and the SELECT statements include joins, grouping, ordering, subqueries, CTEs, and window functions. The goal was not only to prove that the schema works, but also to show that the database can answer realistic Formula 1 analytics questions on the full 700,000+ record dataset.

TABLE IV SQL QUERY COVERAGE

Query	SQL feature	Purpose
Q1	INSERT	Adds a temporary test driver for safe testing.
Q2	INSERT + subquery	Adds a race result by dynamically looking up driverId and statusId.
Q3	DELETE + cascade	Deletes the test driver and verifies dependent result cleanup.
Q4	DELETE	Removes unrealistic pit-stop sensor records above 120000 ms.
Q5	UPDATE + subquery	Assigns surrogate numbers to historical drivers with missing numbers.
Q6	UPDATE + subquery	Zeros points for disqualified race results.
Q7	JOIN, GROUP BY, ORDER BY	Computes all-time driver career statistics.
Q8	CTE + RANK()	Finds the dominant constructor in each decade.
Q9	JOIN, HAVING	Measures qualifying-to-race position gain or loss.
Q10	Four-table JOIN	Analyzes pit-stop efficiency by constructor and season.
Q11	Window SUM + RANK	Builds running 2021 championship standings.
Q12	JOIN + ORDER BY	Tracks one driver championship progression across races.

A. Insert, Delete, and Update Queries

Q1 and Q2: Add a Test Driver and Race Result

The first two queries create a safe temporary driver and attach a sample race result to that driver. This lets us test inserts, triggers, and cascade behavior without changing any real historical driver record. Q2 also uses subqueries, so the database looks up the new driverId and the Finished statusId instead of hardcoding them.

```
INSERT INTO drivers (driverRef, number, code, forename,
  surname, dob, nationality, url)
VALUES ('test_driver', 99, 'TST', 'Test', 'Driver',
  '2000-01-15', 'British',
  'http://en.wikipedia.org/wiki/Test_Driver');

INSERT INTO results (raceId, driverId, constructorId,
  number,
  grid, position, positionText, positionOrder, points,
  laps, statusId)
VALUES (1,
  (SELECT driverId FROM drivers
   WHERE driverRef = 'test_driver'),
  1, 99, 20, 20, '20', 20, 0.00, 50,
  (SELECT statusId FROM status
   WHERE status = 'Finished'));
```

Q3 and Q4: Delete Test Data and Bad Sensor Data

Q3 deletes the temporary driver created for testing. Because the schema uses ON DELETE CASCADE where appropriate, the related test result is also removed automatically. Q4 removes pit stop durations greater than 120 seconds, which are treated as sensor errors or abnormal entries that would distort average pit stop analysis.

```
DELETE FROM drivers
WHERE driverRef = 'test_driver';

DELETE FROM pit_stops
WHERE milliseconds > 120000;
```

Q5 and Q6: Update Historical Driver Numbers and Disqualified Results

Q5 assigns surrogate numbers to historical drivers whose number field is NULL, which makes reporting easier for drivers who raced before permanent numbers were introduced. Q6 keeps disqualified race records in the database but sets their points to zero, which is closer to how F1 stewarding decisions are handled in practice.

```
UPDATE drivers
SET number = driverId + 800
WHERE number IS NULL
  AND driverId IN (SELECT DISTINCT driverId FROM
  results);

UPDATE results
SET points = 0.00
WHERE statusId = (
  SELECT statusId FROM status
  WHERE status = 'Disqualified')
  AND points > 0;
```

B. Analytical SELECT Queries

Q7: All-Time Driver Career Statistics

Q7 joins drivers and results, groups all race entries by driver, and ranks drivers using career points. It also calculates wins, podiums, and average points per race, making it a useful all-time driver comparison query.

```
SELECT d.forename || ' ' || d.surname AS driver_name,
       d.nationality,
       COUNT(r.resultId) AS races_entered,
       SUM(r.points) AS total_points,
       SUM(CASE WHEN r.position = 1 THEN 1 ELSE 0 END) AS wins,
       SUM(CASE WHEN r.position <= 3 THEN 1 ELSE 0 END) AS
  podiums,
       ROUND(AVG(r.points), 2) AS avg_points_per_race
FROM drivers d
JOIN results r ON d.driverId = r.driverId
GROUP BY d.driverId, d.forename, d.surname, d.nationality
ORDER BY total_points DESC
LIMIT 20;
```

Q8: Constructor Dominance by Decade

Q8 uses a CTE and the RANK() window function to find the constructor with the highest total points in each decade. This query is useful because F1 history is often discussed by team-dominance eras rather than by isolated seasons.

```
WITH decade_totals AS (
  SELECT (FLOOR(ra.year / 10.0) * 10)::INT AS decade,
         co.name AS constructor_name,
         SUM(cr.points) AS total_points
  FROM constructor_results cr
  JOIN races ra ON cr.raceId = ra.raceId
  JOIN constructors co ON cr.constructorId = co.constructorId
  WHERE cr.points IS NOT NULL
  GROUP BY (FLOOR(ra.year / 10.0) * 10)::INT,
           co.constructorId, co.name)
SELECT decade, constructor_name, total_points
FROM (
  SELECT decade, constructor_name, total_points,
         RANK() OVER (PARTITION BY decade
                      ORDER BY total_points DESC) AS rnk
  FROM decade_totals) ranked
WHERE rnk = 1
ORDER BY decade;
```

Q9: Qualifying-to-Race Position Gain/Loss

Q9 compares qualifying position with final race position. A positive value shows average places gained in the race, while a negative value shows lost places. The HAVING clause removes drivers with fewer than five races of data so that one-off entries do not dominate the result.

```
SELECT ra.year, d.forename || ' ' || d.surname AS driver_name,
       ROUND(AVG(q.position::NUMERIC - r.positionOrder), 2)
         AS avg_positions_gained,
       COUNT(*) AS races_with_data
FROM qualifying q
JOIN results r ON q.raceId = r.raceId
              AND q.driverId = r.driverId
JOIN drivers d ON q.driverId = d.driverId
JOIN races ra ON q.raceId = ra.raceId
WHERE q.position IS NOT NULL
  AND r.positionOrder IS NOT NULL
GROUP BY ra.year, d.driverId, d.forename, d.surname
HAVING COUNT(*) >= 5
ORDER BY ra.year DESC, avg_positions_gained DESC
LIMIT 30;
```

Q10: Pit Stop Efficiency by Constructor and Season

Q10 computes total stops, average stop time, and fastest stop time per constructor per season. The milliseconds filter removes impossible or abnormal values, and the four-table join connects pit stops back to races, results, and constructors.

```
SELECT ra.year, co.name AS constructor_name,
       COUNT(ps.milliseconds) AS total_stops,
       ROUND(AVG(ps.milliseconds) / 1000.0, 3) AS avg_stop_sec,
       ROUND(MIN(ps.milliseconds) / 1000.0, 3) AS
  fastest_stop_sec
FROM pit_stops ps
JOIN races ra ON ps.raceId = ra.raceId
JOIN results re ON ps.raceId = re.raceId
               AND ps.driverId = re.driverId
JOIN constructors co ON re.constructorId = co.constructorId
WHERE ps.milliseconds BETWEEN 1500 AND 60000
GROUP BY ra.year, co.constructorId, co.name
ORDER BY ra.year DESC, avg_stop_sec ASC
LIMIT 30;
```

Q11 and Q12: Championship Progression

Q11 reconstructs running championship standings for the 2021 season using a cumulative SUM window function and a RANK window function. Q12 then focuses on one driver, driverId = 1, and retrieves that driver championship progression from driver_standings joined with races.

```
WITH cumulative AS (
  SELECT r.driverId, ra.round, ra.name AS race_name,
         r.points AS race_points,
         SUM(r.points) OVER (
           PARTITION BY r.driverId
           ORDER BY ra.round
```

```
          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
          AS cumulative_points
  FROM results r
  JOIN races ra ON r.raceId = ra.raceId
  WHERE ra.year = 2021)
SELECT c.round, c.race_name,
       d.forename || ' ' || d.surname AS driver_name,
       c.race_points, c.cumulative_points,
       RANK() OVER (PARTITION BY c.round
                    ORDER BY c.cumulative_points DESC)
         AS championship_position
FROM cumulative c
JOIN drivers d ON c.driverId = d.driverId
ORDER BY c.round, championship_position;

SELECT r.year, r.name AS race_name,
       ds.points AS cumulative_points,
       ds.position AS championship_position,
       ds.wins
FROM driver_standings ds
JOIN races r ON ds.raceId = r.raceId
WHERE ds.driverId = 1
ORDER BY r.year, r.round;
```

VI. STORED PROCEDURES AND FUNCTIONS

The procedures.sql file defines reusable PostgreSQL functions for the operations that are likely to be used repeatedly. This keeps business logic inside the database and avoids rewriting long SQL statements in every query or application call.

TABLE V STORED FUNCTIONS

Function	Purpose
add_race_result(...)	Inserts one result row and returns the generated resultId.
update_driver_standings(raceId, driverId)	Recomputes cumulative points and wins, then upserts driver standings.
delete_driver(driverId)	Uses a guard clause to refuse deletion if the driver already has results.
get_driver_career_stats(driverId)	Returns career races, points, wins, podiums, and seasons for one driver.
get_constructor_season_summary(constructorId, year)	Returns race-by-race constructor points and standing for a selected season.

For example, get_driver_career_stats(30) returns a career summary for Michael Schumacher, while get_constructor_season_summary(constructorId, year) gives a season-level breakdown for a selected constructor.

```
SELECT * FROM get_driver_career_stats(30);
SELECT * FROM get_constructor_season_summary(131, 2021);
```

VII. TASK 6: TRANSACTIONS AND TRIGGERS

A. Audit and Log Tables

Before writing the triggers, we created three log tables to capture what the triggers do. This gives us a way to verify trigger behaviour and also serves as a basic audit trail for the database: result_audit_log records every INSERT or UPDATE on the results table with a timestamp; driver_deletion_log keeps the deleted driver name and deletion timestamp; transaction_error_log stores failed transaction messages so nothing is silently lost.

B. Triggers Implemented

We implemented six triggers covering the three main scenarios: audit logging so we know what changed, validation so illegal data gets blocked before it reaches the table, and automation so related tables stay in sync without the caller having to think about it. Table VI summarizes all six.

TABLE VI TRIGGERS SUMMARY

Trigger	Event	Timing	Action
trg_results_audit	INS/UPD on results	AFTER	Writes to result_audit_log
trg_results_validate_points	INS on results	BEFORE	RAISE EXCEPTION if points > 26
trg_drivers_deletion_log	DEL on drivers	AFTER	Records in driver_deletion_log
trg_pit_stops_validate	INS on pit_stops	BEFORE	RAISE EXCEPTION if stop > 10
trg_results_auto_standings	INS on results	AFTER	Calls update_driver_standings()
trg_results_no_negative_points	UPD on results	BEFORE	RAISE EXCEPTION if points < 0

Trigger 1: Results Audit Log

trg_results_audit fires AFTER every INSERT or UPDATE on the results table. It writes the operation type, the resultId, driverId, raceId, points value, and a timestamp into result_audit_log. The AFTER timing is intentional - the trigger only fires if the row was actually written, so the audit log is a faithful record of what really changed. If the transaction rolls back later, the audit entry is rolled back with it.

```
CREATE OR REPLACE FUNCTION trg_audit_result()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
INSERT INTO result_audit_log
(action, resultId, driverId, raceId, points)
VALUES
(TG_OP, NEW.resultId, NEW.driverId,
 NEW.raceId, NEW.points);
RETURN NEW;
END;
$$;
```

Trigger 2: Points Validation on Insert

trg_results_validate_points fires BEFORE any INSERT into results. The modern F1 scoring system awards a maximum of 25 points for a race win plus 1 point for the fastest lap, giving an absolute ceiling of 26. If the incoming NEW.points value exceeds 26, the trigger raises an exception immediately and the row is never written. This is the right place to enforce this rule because it applies regardless of which application or script is inserting the row.

```
CREATE OR REPLACE FUNCTION trg_validate_points()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
IF NEW.points > 26 THEN
RAISE EXCEPTION
'Invalid points: % -- max allowed is 26.',
NEW.points;
END IF;
RETURN NEW;
END;
$$;
```

Trigger 3: Driver Deletion Log

trg_drivers_deletion_log fires AFTER a row is deleted from drivers. It writes the deleted driverId, the driver full name (forename + surname), and the deletion timestamp into driver_deletion_log. Because it fires AFTER, the driver row still exists at the moment the function runs - we can still read OLD to get the name before it disappears. This gives a permanent record of who was removed and when, even though the actual driver row is gone.

```
CREATE OR REPLACE FUNCTION trg_log_driver_deletion()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
INSERT INTO driver_deletion_log
(driverId, driver_name)
VALUES
(OLD.driverId,
 OLD.forename || ' ' || OLD.surname);
RETURN OLD;
END;
$$;
```

Trigger 4: Pit Stop Number Validation

trg_pit_stops_validate fires BEFORE any INSERT into pit_stops. It checks whether the stop number exceeds 10. In practice, no F1 driver has made more than 10 pit stops in a single race - anything above that is almost certainly a data entry error or a misclassified event such as a formation lap stop. Catching this BEFORE the row is written means the database never accumulates unrealistic stop counts that would skew pit stop analysis.

```
CREATE OR REPLACE FUNCTION trg_validate_pit_stop()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
IF NEW.stop > 10 THEN
RAISE EXCEPTION
'Invalid stop number: % for driver % '
'in race % -- max 10 allowed.',
NEW.stop, NEW.driverId, NEW.raceId;
END IF;
RETURN NEW;
END;
$$;
```

Trigger 5: Automatic Standings Update

trg_results_auto_standings fires AFTER every INSERT into results. It calls update_driver_standings() with the raceId and driverId from the new row. That function recomputes cumulative points and wins for that driver up to that race and upserts the result into driver_standings. This means the standings table is always in sync with results - nobody has to remember to update standings separately after inserting a result. It is an example of automation: the caller inserts one row and the trigger handles the downstream bookkeeping.

```
CREATE OR REPLACE FUNCTION trg_auto_update_standings()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
PERFORM update_driver_standings(
```

```
NEW.raceId, NEW.driverId);
RETURN NEW;
END;
$$;
```

Trigger 6: Prevent Negative Points on Update

trg_results_no_negative_points fires BEFORE any UPDATE on results. It checks whether NEW.points would go below zero. Championship points can never be negative in F1 - the minimum is zero, which applies to a driver who did not score. If a stewards penalty or a data correction would push points negative, this trigger blocks it with an exception. This is separate from Trigger 2 because that one handles INSERT; this one handles UPDATE, specifically the case where points are being edited on an existing row.

```
CREATE OR REPLACE FUNCTION trg_prevent_negative_points()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
IF NEW.points < 0 THEN
RAISE EXCEPTION
'Points cannot be negative: tried to set '
'points = % for resultId %.',
NEW.points, NEW.resultId;
END IF;
RETURN NEW;
END;
$$;
```

C. Transaction Failure Demonstration

To show that atomicity actually works the way PostgreSQL promises, we set up a scenario where a batch of two race results is submitted and the second one is deliberately invalid - points = 100, which exceeds the modern maximum of 26. What we want to confirm is that neither result makes it into the database, not even the first valid one, and that the error gets logged cleanly.

```
DO $$
BEGIN
-- Step 1: valid result (25 pts) -- passes
-- trg_results_validate_points: 25<=26 OK
-- trg_results_audit: row written to audit log
-- trg_results_auto_standings: standings upserted
INSERT INTO results (raceId, driverId,
constructorId, grid, positionOrder,
points, laps, statusId)
VALUES (1, 1, 1, 1, 1, 25.00, 58, 1);
RAISE NOTICE 'Step 1 succeeded.';

-- Step 2: invalid (points=100) -- trigger fires
-- trg_results_validate_points: 100>26
-- => RAISE EXCEPTION
-- All of Step 1 (insert + audit + standings)
-- is ROLLED BACK
INSERT INTO results (raceId, driverId,
constructorId, grid, positionOrder,
points, laps, statusId)
VALUES (2, 2, 1, 2, 2, 100.00, 58, 1);
EXCEPTION WHEN OTHERS THEN
-- Runs in a sub-context; persists after rollback
INSERT INTO transaction_error_log(error_msg)
VALUES ('Aborted: ' || SQLERRM);
RAISE NOTICE 'Rolled back: %', SQLERRM;
END;
$$;
```

D. What Happens When a Transaction Is Aborted

Here is what actually happens step by step when the DO block runs: 1. Step 1 inserts a valid result (25 pts). The BEFORE trigger checks 25 <= 26 and passes. The row is written. The AFTER audit trigger fires and logs it. The standings trigger updates driver_standings. 2. Step 2 attempts to insert points = 100. The BEFORE trigger fires and raises an exception immediately - the INSERT never completes. 3. PostgreSQL rolls back everything to an implicit savepoint: the Step 1 result row, the audit log entry, and the standings upsert all disappear as if they never happened. 4. The EXCEPTION block catches the error and logs it to transaction_error_log. This runs in its own clean context so the error record does persist even though the rest was rolled back.

This behaviour is exactly what you want for a race-result load. If one entry in a batch is wrong, you do not want half the results committed and half missing - that would produce incorrect championship standings. Rolling back and logging the error lets the operator fix the bad entry and resubmit the whole batch.

```
-- Must return 0 (Step 1 was rolled back)
SELECT COUNT(*) FROM result_audit_log;
-- Must return 1 (error was captured)
SELECT * FROM transaction_error_log;
```

VIII. TASK 7: INDEXING AND QUERY EXECUTION ANALYSIS

After the queries were working correctly, we ran EXPLAIN ANALYZE on the ones that touched the largest tables to see where time was actually being spent. Three of them showed obvious problems - full sequential scans on tables with tens of thousands of rows, joined together with hash joins. We identified the right indexes for each case and measured the improvement.

A. Problematic Query 1: Pit Stop Efficiency Join (Q10)

Problem: Q10 joins four tables - pit_stops, races, results, and constructors. Without indexes, the planner falls back to a full sequential scan on results (26,759 rows) and a nested-loop materialize that generates 981,655 intermediate rows before the join filter removes them. The 552 kB in-memory materialize buffer and repeated looping over rows account for most of the cost.

EXPLAIN ANALYZE before indexing (actual output)

```
Nested Loop (cost=1.01..2909.22
rows=1774)
(actual time=37.109..78.148 rows=3019)
-> Seq Scan on results re
(cost=0.00..583.59 rows=26759)
(actual time=0.044..2.565 rows=26760 loops=1)
-> Materialize
(cost=0.30..2147.49 rows=1734)
Storage: Memory Max: 552kB
Rows Removed by Join Filter: 981655
-> Index Scan using pit_stops_pkey
on pit_stops
Filter: milliseconds BETWEEN 1500 AND 60000
Planning Time: 2.877 ms
Execution Time: 79.135 ms
```

The sequential scan on results reads all 26,759 rows on every outer loop iteration. The join filter then discards 981,655 combinations before producing the final 3,019 rows. This is extremely wasteful - we only need rows where raceId and driverId match between the two tables.

Indexes created

```
CREATE INDEX IF NOT EXISTS
idx_results_raceid_driverid
ON results (raceId, driverId);
CREATE INDEX IF NOT EXISTS
idx_pit_stops_raceid_driverid_ms
ON pit_stops (raceId, driverId, milliseconds);
-- Index creation took 7.927 seconds (one-time cost)
```

EXPLAIN ANALYZE after indexing (actual output)

```
Nested Loop
(cost=0.72..31809.97
rows=1699)
(actual time=0.156..8.246 rows=3019)
-> Index Only Scan using
idx_pit_stops_raceid_driverid_ms
on pit_stops
Index Cond: (raceid = ra.raceid)
AND (milliseconds >= 1500)
AND (milliseconds <= 60000)
-> Index Scan using
idx_results_raceid_driverid
on results re
Index Cond: (raceid = ps.raceid)
AND (driverid = ps.driverid)
Planning Time: 4.203 ms
Execution Time: 11.123 ms
```

Result: The composite index on pit_stops covers both the join key and the milliseconds range filter in a single Index Only Scan, eliminating the Seq Scan and the 552 kB materialize entirely. The results index replaces the nested loop over all 26,759 rows with a direct Index Scan per pit stop row. Execution time dropped from 79.135 ms to 11.123 ms - a real 7.1x improvement measured on the live database.

B. Problematic Query 2: Driver Standings Lookup (Q12)

Problem: Q12 filters the 34,863-row driver_standings table by a single driverId. Without an index, PostgreSQL must read every row in the table and discard the ones that do not match.

EXPLAIN ANALYZE before indexing (actual output)

```
Sort
(cost=765.72..766.63 rows=361)
(actual time=3.737..3.756 rows=356)
-> Hash Join
(cost=716.18..750.39 rows=361)
(actual time=3.258..3.568 rows=356)
-> Seq Scan on races r
(actual time=0.008..0.121 rows=1125 loops=1)
-> Hash
Memory Usage: 27kB
-> Seq Scan on driver_standings ds
(cost=0.00..711.66 rows=361)
(actual time=0.018..3.155 rows=356 loops=1)
Filter: (driverid = 1)
Rows Removed by Filter: 34507
Planning Time: 3.939 ms
Execution Time: 3.823 ms
```

The sequential scan reads all 34,863 rows and discards 34,507 of them to return 356 matching rows for Hamilton. A 27 kB hash is then built over the race table to execute the join. Every new season adds roughly 860 rows (one per driver per race), so this cost grows linearly with the dataset.

Index created

```
CREATE INDEX idx_driver_standings_driverid
ON driver_standings(driverId);
```

EXPLAIN ANALYZE after indexing (actual output)

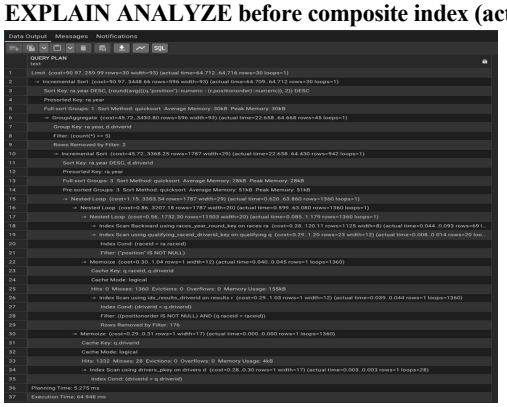
```
Sort
(cost=342.93..343.81 rows=351)
(actual time=0.845..0.857 rows=356)
-> Hash Join
(cost=293.88..328.10 rows=351)
(actual time=0.536..0.726 rows=356)
-> Seq Scan on races r
(actual time=0.006..0.079 rows=1125 loops=1)
-> Hash Memory Usage: 27kB
-> Bitmap Heap Scan on driver_standings ds
(cost=7.01..289.49 rows=351)
(actual time=0.082..0.435 rows=356 loops=1)
Heap Blocks: exact=62
-> Bitmap Index Scan on
idx_driver_standings_driverid
(actual time=0.065..0.065 rows=356 loops=1)
Index Cond: (driverid = 1)
Planning Time: 1.021 ms
Execution Time: 0.922 ms
```

Result: The Bitmap Index Scan jumps directly to the 356 matching rows and reads only 62 heap blocks instead of the entire table. Execution time dropped from 3.823 ms to 0.922 ms - a 4.1x improvement. The gain becomes more significant as new seasons are added, since the index cost stays nearly constant while a sequential scan grows linearly with table size.

C. Problematic Query 3: Qualifying vs Race Position (Q9)

Problem: Q9 joins qualifying (10,494 rows) to results (26,759 rows) on (raceId, driverId). Before the composite index existed, the planner had a partial index on results(driverId) only. It used that index to look up by driverId, then had to apply a secondary filter for raceId on each match - removing 176 rows per inner loop iteration across 1,360 loops. That secondary filter work was entirely avoidable.

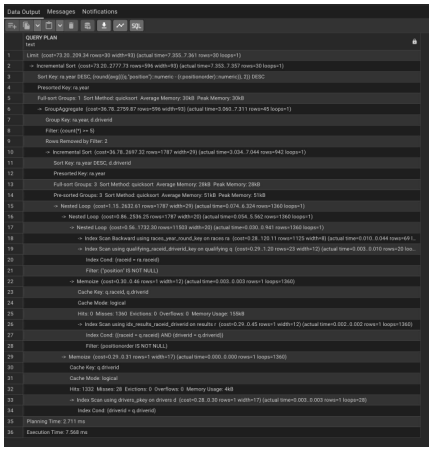
EXPLAIN ANALYZE before composite index (actual output)



With 1,360 inner loop iterations each discarding 176 rows from the driverId-only index scan, the planner was doing 1360 x 176 = 239,360 unnecessary row comparisons. The driverId index narrows the candidate set but cannot eliminate the raceId mismatch without also indexing that column.

Index used: idx_results_raceid_driverid (raceId, driverId) - already created for Q10. Zero additional storage cost, and it now covers both join columns.

EXPLAIN ANALYZE after composite index (actual output)



Result: The composite index satisfies both join conditions directly. The secondary raceId filter disappears from the inner loop, and actual time per inner iteration drops from 0.039-0.044 ms to 0.002 ms. Total execution time fell from 64.948 ms to 7.568 ms - an 8.6x improvement, and the same index already served Q10, so no extra storage was needed.

D. Summary of Index Improvements

TABLE VII QUERY PERFORMANCE BEFORE AND AFTER INDEXING

Query	Before	After	Speedup
Q10 Pit Stop Join	79.135 ms	11.123 ms	7.1x
Q9 Qualifying Join	64.948 ms	7.568 ms	8.6x
Q12 Standings Lookup	3.823 ms	0.922 ms	4.1x

Scalability Note: All three speedups were measured on the live database. Q12s 4.1x gain will grow over time - the index cost stays nearly constant while a sequential scan grows linearly with each new season. The 589,081-row lap_times table is the next bottleneck: a composite index on

(raceId, driverId) is the obvious next step for any lap-time analytics queries.

7. Bonus Task: Interactive Dashboard

DashboardLink:
https://public.tableau.com/app/profile/sphoorthy.selvaraj/viz/F1RaceAnalytics-CSE560/F1RaceAnalyticsDashboard

We built a Tableau Public dashboard with five visualizations drawn from our F1 database. The CSV files were connected in Tableau using foreign key relationships (driverId, raceId, constructorId, circuitId), with filters applied for race wins (position = 1) and pit stop outliers (milliseconds <= 60,000).

SQL Queries

Q1 — Top 10 Drivers by Career Wins

```
SELECT d.forename || ' ' || d.surname AS driver, COUNT(*) AS wins
FROM results r JOIN drivers d ON r.driverId = d.driverId
WHERE r.position = 1
GROUP BY d.driverId, d.forename, d.surname
ORDER BY wins DESC LIMIT 10;
```

Q2 — Wins by Driver Nationality

```
SELECT d.nationality, COUNT(*) AS wins
FROM results r JOIN drivers d ON r.driverId = d.driverId
WHERE r.position = 1
GROUP BY d.nationality ORDER BY wins DESC LIMIT 12;
```

Q3 — Constructor Points by Decade

```
SELECT (r.year / 10) * 10 AS decade, c.name,
ROUND(SUM(res.points)::numeric, 0) AS total_points
FROM results res JOIN races r ON res.raceId = r.raceId
JOIN constructors c ON res.constructorId = c.constructorId
WHERE c.name IN ('Ferrari','Mercedes','McLaren','Williams','Red Bull','Renault')
GROUP BY decade, c.name ORDER BY decade, total_points DESC;
```

Q4 — Pit Stop Duration Trends

```
SELECT r.year, ROUND(AVG(ps.milliseconds)::numeric / 1000, 2) AS avg_sec
FROM pit_stops ps JOIN races r ON ps.raceId = r.raceId
WHERE ps.milliseconds <= 60000
GROUP BY r.year ORDER BY r.year;
```

Q5 — Most-Raced Circuits

```
SELECT ci.name, ci.country, COUNT(*) AS race_count
FROM races r JOIN circuits ci ON r.circuitId = ci.circuitId
GROUP BY ci.circuitId, ci.name, ci.country
ORDER BY race_count DESC LIMIT 10;
```

Dashboard Design

The dashboard uses a tiled two-row layout. The top row shows Constructor Points by Era (stacked bar) and Wins by Nationality (vertical bar). The bottom row holds Top 10 Career Wins (horizontal bar), Pit Stop Trends (line chart), and Top Circuits (horizontal bar). Team colors match real-world branding (Ferrari red, Mercedes teal, McLaren orange). All charts include data labels, and Tableau's hover tooltips provide additional detail.

Insights

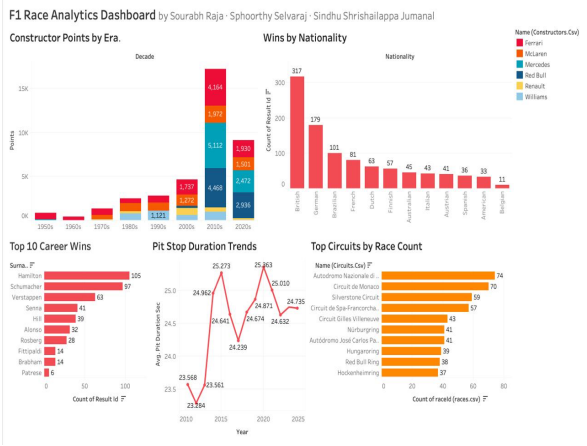
Top 10 Career Wins: Hamilton leads with 105 wins, 8 ahead of Schumacher (97). The top three alone account for 265 wins, reflecting how sustained team partnerships drive career totals.

Wins by Nationality: British drivers hold 317 wins, nearly double Germany's 179. This stems from the UK's motorsport infrastructure — most F1 teams are based in England, giving British drivers more direct pathways into the sport.

Constructor Points by Era: Ferrari is the only team scoring points in every decade since the 1950s. Mercedes and Red Bull emerged in the 2010s but accumulated more points in that single decade than most teams have in their entire history.

Pit Stop Trends: Average durations stayed stable between 23–25 seconds from 2011 to 2024. The slight increase after 2014 coincides with heavier hybrid-era cars requiring adjusted pit procedures.

Top Circuits: Monza (74 races) and Monaco (70) have hosted races almost continuously since 1950. European circuits dominate the top 10, reflecting F1's historical roots.



IX. CONCLUSION

Across both phases, this project built a complete PostgreSQL database over 75 years of Formula 1 history. Phase 1 established the foundation through a BCNF-normalized schema across 14 tables, an ER diagram that reflects the real structure of the data, and a normalization analysis grounded in the actual CSV rows. The most useful finding from that work was that driver_standings and constructor_standings had a subtle BCNF issue involving positionText, which was corrected without losing information.

Phase 2 put the database to work. The twelve SQL queries tested updates and realistic analytics, the stored functions made repeated operations easier to call, and the triggers gave the database its own validation and audit layer. The transaction failure demonstration showed why rollback is necessary when invalid race-result data is submitted. Finally, the indexing analysis showed measurable performance improvements on realistic workloads, with speedups of 4.1x to 8.6x. The schema remains unchanged from Phase 1, all 14 relations remain in BCNF, and the final system supports both reliable updates and meaningful historical F1 analysis.

REFERENCES

[1] Ergast Developer API, "Motor Racing Data," http://ergast.com/mrd/, accessed 2025.

[2] R. Rao, "Formula 1 World Championship (1950-2024)," Kaggle, https ://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-19 50-2020, updated Jan. 2025.

[3] R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed., McGraw-Hill, 2003.

[4] R. Elmasri and S. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[5] IEEE, "Conference Paper Template," https://www.ieee.org/conferences/publishing/templates.html, 2024.